

Autonomous Retrieval for Continuous Learning in Associative Memory Networks

Saighi Paul^{*1}, Rozenberg Marcelo^{1,2},

1 Université Paris-Saclay, CNRS, Laboratoire de Physique des Solides, 91405, Orsay, France

2 Université Paris-Cité, CNRS, Integrative Neuroscience and Cognition Center, 75006, Paris, France

* paul.saighi@universite-paris-saclay.fr

Abstract

The brain's faculty to assimilate and retain information, continually updating its memory while limiting the loss of valuable past knowledge, remains largely a mystery. We address this challenge related to continuous learning in the context of associative memory networks, where the sequential storage of correlated patterns typically requires non-local learning rules or external memory systems. Our work demonstrates how incorporating biologically-inspired inhibitory plasticity enables networks to autonomously explore their attractor landscape. The algorithm presented here allows for the autonomous retrieval of stored patterns, enabling the progressive incorporation of correlated memories. This mechanism is reminiscent of memory consolidation during sleep-like states in the mammalian central nervous system. The resulting framework provides insights into how neural circuits might maintain memories through purely local interactions, and takes a step forward towards a more biologically plausible mechanism for memory rehearsal and continuous learning.

Author summary

Catastrophic forgetting - when acquiring new knowledge seriously degrades previously learned information - remains a fundamental challenge in machine learning, affecting systems from simple associative networks to complex language models. One widely studied approach to mitigate forgetting is memory rehearsal, in which prior stored information are periodically replayed, an idea supported by neurophysiological evidence of memory consolidation during sleep. In this work, we show that networks with adaptation can spontaneously recall stored memories without external guidance. This built-in retrieval mechanism allows for the incorporation of new memories while preserving older ones, opening a path toward a biologically inspired solution to the problem of catastrophic forgetting.

Introduction

Continuous Learning (CL) refers to a system's ability to maintain performance across multiple tasks when operating in environments that evolve over time, requiring adaptation to changing data distributions. To do so, the learning mechanism should avoid uncontrolled forgetting of previously acquired knowledge when adapting to new information or contexts. In associative memory networks, this challenge arises when storing new activity patterns sequentially deteriorates existing memory representations, a phenomenon called catastrophic forgetting.

It is important to note that this work specifically addresses catastrophic forgetting in the context of sequential learning. This approach addresses a different challenge than the well-studied spin glass phase transitions that occur in Hopfield networks at high memory loads.

Memory rehearsal is a method that addresses the challenge of catastrophic forgetting by periodically retraining the model on previously stored patterns. This process reinforces older memory representations, preventing their degradation when new information is incorporated [1, 2]. In the mammalian nervous system, spontaneous memory replays occur during sleep [1, 3–6], suggesting a biological mechanism analogous to rehearsal techniques in artificial networks [1]. This parallel raises a fundamental question: what mechanisms enable these autonomous memory replays in biological systems?

Previous research on bioinspired neural networks demonstrates that short-term synaptic depression can facilitate spontaneous rehearsal of neural assemblies [5]. However, a significant constraint of this approach is its dependence on minimal overlap between neural assemblies. The neuronal populations in these studies share few neurons, resulting in effectively decorrelated memory representations. By contrast, classical associative memory networks like Hopfield Networks can effectively store uncorrelated memories that share many units. Some learning algorithms even allow the storage of highly correlated patterns that share a majority of their units [7]. From a biological perspective, understanding how networks implement the rehearsal of correlated populations is crucial as neural representations found in the cortex generally recruit extensively overlapping assemblies [8, 9]. The maintenance of overlapping assemblies is widely considered essential for cortical computation, as it supports stimulus generalization and the emergence of invariant, high-level concepts in which individual neurons participate in multiple but related representations [8–10]. This problematic is of similar importance in neuromorphic engineering contexts, as highly correlated representations are an emerging feature of artificial neural networks [11].

How the rehearsal of correlated memories takes place autonomously on neural substrates remains a largely unaddressed question. In this work, we focus on this issue and explore its potential application for continuous learning (CL). For the sake of bio-plausibility and potential implementation in neuromorphic substrates, we shall demand that our system exhibits the following features: 1) It stores memory states that can be highly correlated; 2) During the pattern recovery, the network does not converge toward strange attractors which would constitute false memories; 3) Plasticity rules are local, meaning that the modification of synaptic efficacy can be computed in terms of its pre- and post-synaptic neuron states; 4) It is autonomous, namely, it should retrieve all previously stored patterns from its own dynamics. The network does not have access to an external list of previously recorded memory states. For the sake of requirements (1) and (3), we shall adopt a perceptron-like algorithm, inspired by the work of Diederich & Oppen (1987) [7]. On the other hand, for requirement (2) we shall use continuous Hopfield Networks (CHNs) [12]. Our work demonstrates that, kept under a certain memory load, CHNs converge exclusively to stored patterns during the retrieval. This approach avoids both, the spurious state proliferation common in Discrete Hopfield

Networks (DHNs) and the shortcomings of temperature parameter fine-tuning inherent to Stochastic Hopfield Networks (SHNs) [13].

The main contribution of the present work is to introduce an algorithm to address the requirement (4). A crucial feature of our approach is the use of self-inhibition to reduce the attraction toward previously visited attractor states, thus allowing for a sequential and thorough search and recovery of all previously stored correlated memory states.

This recovery effectively allows the rehearsal of the stored pattern for CL purposes. The dynamic of plastic recurrent inhibition is inspired by computational neuroscience work based on actual neurophysiological data [14].

1 Methods

1.1 Continuous Hopfield Network (CHN)

In contrast with the conventional DHN model [15], where neural states are defined as binary variables, in a CHN they are continuous [12]. The dynamics of the network is defined by a set of differential equations with each neuron unit described as a leaky integration:

$$c \frac{du_i}{dt} = \sum_j W_{ij} v_j - \frac{u_i}{r} \quad (1)$$

where u_i is the membrane potential of neuron i , c is the membrane capacitance, r is the leak resistance of each neuron, W_{ij} is the synaptic efficacy between neurons j and i , and v_i is the activity (or firing rate) of neuron i that depends solely on the potential as

$$v_i = \sigma(u_i)$$

where σ is a monotonically increasing function of u with saturation to prevent runaway dynamics. We adopt $\sigma(x) = \frac{1}{1+\exp(-x)}$. Therefore, as $\sigma(0) = 0.5$, each unit has a positive output at the resting state, allowing the network to have a baseline activity without external input. We shall refer to either the vector $\mathbf{v}(t)$ or $\mathbf{u}(t)$ as "states", which should be clear from the context.

The convergence of the flow to stable states for the case of symmetric synaptic weights W_{ij} has been demonstrated [12]. Throughout this work, whenever we integrate these equations using Euler method, we do so until the network reaches convergence, defined as $|\frac{du}{dt}|_\infty < \epsilon$, where $\epsilon = 10^{-6}$.

1.2 Pattern Storage and Reading

The states $\mathbf{v}(t)$ of the CHN evolve on $[0, 1]^N$ through continuous dynamics, with N the number of neurons. Any state in this space may represent a stored pattern. For simplicity, we restrict ourselves to store binary patterns for which active neurons have a high firing rate, $v_i \approx 1$, and inactive neurons have a low firing rate, $v_i \approx 0$.

Hence, a pattern $\mathbf{x}^\mu$ is defined as a binary vector such that $\mathbf{x}^\mu = (x_1^\mu, x_2^\mu, \dots, x_N^\mu)$ where $x_i^\mu \in \{0, 1\}$ for each unit i . A pattern is read from the state of the network at time t using a threshold:

$$x_i^\mu = \begin{cases} 1 & \text{if } v_i(t) > 0.5 \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

Given a binary pattern $\mathbf{x}^\mu$, it is convenient to define target potentials $\tilde{\mathbf{u}}^\mu$ as:

$$\tilde{u}_i^\mu = \begin{cases} +u_{\text{target}}^\mu & \text{if } x_i^\mu = 1 \\ -u_{\text{target}}^\mu & \text{if } x_i^\mu = 0 \end{cases} \quad (3)$$

We adopt here $u_{\text{target}} = 6$ to ensure proper pattern reading following the thresholding procedure.

Inspired by previous work on DHN [7], we introduce, in Alg. 1, a perceptron-inspired learning algorithm for efficient storage of correlated patterns in CHNs. The algorithm minimizes the error between the target states and the network’s equilibrium states, ensuring that each memory becomes a stable state. Gradient descent methods typically require small step sizes to prevent the optimization process from becoming unstable and to reduce oscillations around local minima. In our implementation, we select $\alpha = 0.0001$ as the learning rate to ensure stable convergence of weight updates. The derivation of the weight update rule can be found in the Appendix. 5.1. Although a rigorous proof of convergence for the algorithm is beyond the scope of this work, we expect that arguments demonstrating the convergence of the gradient descent algorithm (GDA) in the context of DHN could be adapted for this purpose [7]. Here, we rely on numerical evidence showing the network’s ability to successfully query and revisit stored patterns.

Algorithm 1 Gradient descent for the storage of correlated patterns (GDA)

```

1: Initialize  $W_{ij} = 0$  for all  $i, j$ 
2: repeat
3:   for each pattern  $\mu$  do
4:     Compute the target potential  $\tilde{u}_j^\mu$  (Eq. 3) for each neuron  $j$  with  $j \neq i$ 
5:     for each neuron  $i$  do
6:       Compute the expected potential:  $\hat{u}_i^\mu = r \sum_j W_{ij} \sigma(\tilde{u}_j^\mu)$ 
7:       Update weights:  $W_{ij} \leftarrow W_{ij} - \alpha(\hat{u}_i^\mu - \tilde{u}_i^\mu)r\sigma(\hat{u}_j^\mu)$ 
8:     end for
9:   end for
10: until  $\|\Delta W\|_\infty < \epsilon$ 

```

Following the network training with the GDA, each activity vector $\mathbf{u}^\mu$ becomes an attractor. The network can now be queried as the system reliably converges to the nearest stored state from a partial cue. The corresponding binary patterns $\mathbf{x}^\mu$ can then be accurately retrieved by thresholding at time t_f (Eq. 3) when the system reaches convergence. The querying procedure is detailed in Alg. 4 (Appendix 5.2) and illustrated in Fig. 1.

Similarly to the model proposed by Diederich and Oppen (1987), the symmetry of the weight matrix W is not enforced in the GDA scheme. In classical Hopfield networks, this constraint is typically imposed to prevent unwanted oscillatory dynamics [15]. A notable feature of the GDA is that it consistently stores patterns as attractor states despite the absence of enforced symmetry [7]. We observed the same phenomenon in our simulations. This represents a strength of the algorithm, as enforcing symmetry without an explicit underlying mechanism would break the locality of the plasticity rules. In biological neural networks, the synapse from neuron A to neuron B does not necessarily have access to the state or sign of the reciprocal synapse from neuron B to A.

It is worth emphasizing that despite the continuous nature of our model, which theoretically allows for a richer state space, we deliberately restrict ourselves to binary patterns. The interest of using a CHN is the simplicity it allows when implementing our pattern recovery mechanism (Sec. 1.4), limiting the appearance of false memories as spurious states, which are often encountered in DHNs [16]. A variant of the DHN with stochastic units, the SHN [16], would be a possible candidate to implement our algorithm, as they tend to visit only the stored patterns by properly controlling the annealing temperatures. However, the stochastic properties of these networks would require a more complex setup.

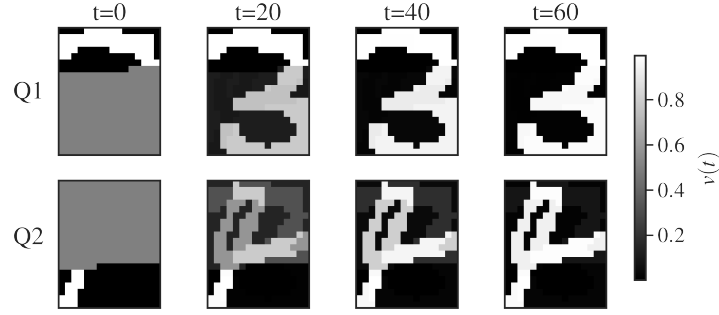


Fig 1. Querying of two binary-patterns representation of the handwritten digit "3" and "4" from the MNIST dataset. The network is of size 20×16 as each neuron codes for a pixel. From black to white, the color represents the rate $v_i(t)$ of each unit. At $t = 0$ the network is initialized with the part of the units set to the target value as described in Appendix 5.2. The figure displays snapshots of the evolution of the rates in time for each unit of the network. The network is queried two times, for the first query Q1 the network gradually settles to the nearest stored state corresponding to the digit "3". For Q2, the network settles to the stored state corresponding to the digit "4".

1.3 Continuous incorporation of correlated memories

While the GDA effectively enables the storage of correlated memories, its implementation in Alg. 1 reveals a significant limitation: it requires multiple iterations over the entire set of patterns to achieve convergence. Without the ability to reprocess all patterns, the network would suffer from catastrophic forgetting, where learning a new pattern in isolation rapidly erodes previously stored memories [1, 17–19]. By repeatedly processing all patterns, the algorithm can find a weight matrix $\mathbf{W}$ that properly separates the patterns, despite their correlations [7]. Adding a new pattern, therefore, requires access to all previously stored patterns from an external source.

This requirement for external access to the complete memory dataset stands in contrast to biological learning systems, which must incorporate new information while maintaining past memories without relying on an explicit external copy of the already stored data. To overcome this external dependency and move towards more biologically plausible learning, a solution is to develop a mechanism that allows the network to internally recover its stored memories.

The development of such an autonomous retrieval mechanism would allow us to exploit the GDA's ability for the continuous incorporation of correlated memories. The continuous learning algorithm is formally defined in Alg. 2. The act of retraining the network on the whole set is called a rehearsal. Recovering the whole set from the network to allow rehearsal is called retrieval.

Algorithm 2 Continuous incorporation of correlated patterns through rehearsal of the whole memory set

- 1: **Input:** CHN trained on p patterns using the GDA
 - 2: **Given:** New patterns $\{\mathbf{x}^{p+1}, \mathbf{x}^{p+2}, \dots\}$ to be stored
 - 3: Retrieve set $\{\mathbf{x}\}$ of stored patterns through autonomous retrieval (AR) introduced in Sec. 1.4
 - 4: Update pattern set: $\{\mathbf{x}\} \leftarrow \{\mathbf{x}\} \cup \{\mathbf{x}^{p+1}, \mathbf{x}^{p+2}, \dots\}$
 - 5: Apply GDA to store updated pattern set
-

1.4 Autonomous retrieval

In this section, we present the autonomous retrieval (AR) mechanism allowing the recovery of stored correlated patterns in a network.

Given a trained network initialized at the "neutral" state, $u_i = 0$ for each unit i , the network dynamics described by Eq. 1 converge deterministically to a given stored attractor, which is thus "retrieved". This attractor can be seen as the dominant attractor from the neutral state. The goal now is to allow for the exploration of other states to permit a complete recovery of the stored memories. To do so, we introduce the adaptation terms A_i .

$$c \frac{du_i}{dt} = \sum_j W_{ij} v_j - A_i v_i - \frac{u_i}{r} \quad (4)$$

$$A_i \leftarrow A_i + \beta v_i(t_f) \quad (5)$$

with $v_i(t_f)$ the firing rate of neuron i after convergence of the dynamics. β represents the adaptation strength and is chosen to be small, typically below 0.1. Adaptation terms could correspond to various components commonly observed in the mammalian central nervous system. On short time scales, spike frequency adaptation (SFA) of excitatory neurons functions as plastic self-inhibition [20, 21]. Repeated stimulation progressively reduces firing activity in the neuron, mirroring the dynamics produced by our adaptation A_i . Alternatively, A_i can be interpreted as recurrent inhibition mediated by local inter-neurons, which frequently exhibit Hebbian plasticity [22, 23].

After a given memory is retrieved (i.e., when the network falls into an attractor), the attraction toward that state is decreased through the adaptation term (Eq. 5). Consequently, once the pattern has been inhibited, the probability for the network to converge into it from the neutral state is reduced. By resetting the network to the neutral state after each convergence-inhibition cycle, we allow the sequential recovery of stored patterns.

Fig. 2 illustrates the sequential recovery of two stored patterns and the modification of the network dynamics induced by the procedure. The geometry of the vector field explains why a minimal inhibitory influence, resulting from a small β value, is sufficient to alter the trajectory. The neutral state resides near the separatrix that divides the attractor basins corresponding to each patterns. Consequently, even a slight modification of the separatrix position, caused by inhibitory potentiation, can significantly redirect the network's trajectory.

The increase in adaptation tends to distort the stable states associated with stored patterns. As this distortion is minimal for small β , it is mainly compensated for by the thresholding mechanism used to read the network output (Sec. 1.2). To further reduce the impact of this distortion, we divide the convergence dynamic into two phases, the "biased" phase and the "free" phase. The biased phase guides the network to converge toward states that have not been retrieved. It corresponds to the simulation of the network with adaptation, Eq. 4, until convergence. The optional free phase then allows the network to complete its convergence to an undistorted stored state. It corresponds to the simulation of the network without inhibitory synapses (Eq. 1) until convergence. The whole procedure is detailed in Alg. 3.

It is important to note that correlated patterns and patterns with various sparsity levels induce similar vector fields. In all cases, we observe a separatrix crossing the neutral state and lying in the middle of the attraction basins. It appears to be an emergent property of the gradient descent algorithm, appropriately shaping the dynamical landscape to balance the attraction strength of memory states from the neutral state. The construction of correlated patterns is discussed in Sec. 2. The construction of patterns with various sparsity levels is discussed in Sec. 3.

Fig. 3 provides a visualization of AR for binary-patterns representation of handwritten digits from the MNIST dataset [24]. For the first iteration, the inhibitory drive is null as no pattern has been retrieved. The pattern corresponding to the binary picture of a 3 is recovered. The network state is reinitialized with the updated adaptation (**A**). The network now inhibits the recovery of a 3, which induces convergence toward a second pattern, here a 4. Adaptation is updated again and now inhibits both the 3 and the 4 together. Inhibition of the 3 and the 4 combined allows the recovery of the 5 and so on. For the recovery of MNIST binary digits, as a large inhibitory coefficient β has been chosen, the free phase is mandatory to reduce the distortion of the stored pattern.

The order in which patterns will be visited is an emerging feature of the learning algorithm that has not been studied in this work. With each iteration of the AR algorithm, inhibitory synapses undergo potentiation. This growth is constrained only by the number of iterations and the value of β . Theoretically, such an unbounded growth could cause the adaptation A_i to disrupt the attractor dynamics induced by $\mathbf{W}$. However, in practice, this potential issue can be managed by adjusting the value of β based on the number of iterations. Specifically, when more iterations are required, using a smaller β is sufficient to maintain robust pattern retrieval without compromising attractor dynamics.

In Fig. 4, we illustrate the dynamics of the CHN during the retrieval of stored memory patterns in networks with various loads. The load corresponds to M/N , with M the number of stored memories, and N the size of the network. For low loads (see Fig. 4(top)), the system converges sequentially to the attractors corresponding to the stored patterns. Once every memory has been recovered, if the simulation is not ended, the network falls back into the stored states without showing any false memories. The lower the value of β , the longer this dynamic can proceed without encountering spurious states. The free phase has no utility in this scenario, the attractors are well separated, and the disruption of the energy landscape by adaptation is minimal. For critical loads (see Fig. 4(middle)), the retrieval process becomes more challenging. The attractors exhibit reduced separation, and the network struggles to converge to the stored states once inhibition is applied. In this scenario, the free phase demonstrates its utility. At the end of the biased phase, during the sixth iteration of the AR, the network remains in a mixed state characterized by ambiguous correlations with multiple stored patterns. The free phase enables the network to resolve this ambiguity and ultimately converge to a stored state. At high loads (see Fig. 4(bottom)), the network successfully retrieves some stored states, but mostly falls into spurious attractors. Under these conditions, even the free phase cannot rescue the recovery dynamics.

These autonomous recovery dynamics are reminiscent of memory replays observed in the central nervous system of mammals during quiescent states, such as sleep or rest. This process is believed to function as a consolidation mechanism that mitigates or modulates memory forgetting [1, 3–6].

The use of recurrent plastic inhibitory synapses has been tested and shows the same qualitative dynamics as that observed for networks with units that undergo adaptation. The results and the model are detailed in App. 5.4. As implementing adaptation requires less computation and parameters, the following work focuses only on the adaptation model.

Algorithm 3 Autonomous retrieval (AR)

```

1: Input: Trained network weights  $W_{ij}$ , number of iterations  $k$ , plasticity rate  $\beta$ 
2: Initialize:  $A_i = 0$ ,  $\{\mathbf{x}\} = \emptyset$ 
3: while  $j < k$  do
4:   Set neutral initial conditions:  $\mathbf{u}(t=0) = \mathbf{0}$ 
5:   Biased phase: Integrate Eq. 4 until convergence to the state  $\mathbf{u}(t_b) = \mathbf{u}_b$ 
6:   Free phase: from the state  $\mathbf{u}(t_b)$ , integrate Eq. 1 until convergence  $\triangleright$  optional
7:   Read pattern  $\mathbf{x}^\mu$  from final state  $\mathbf{v}(t_f)$  via thresholding (Eq. 3)
8:   Update retrieved pattern set:  $\{\mathbf{x}\} \leftarrow \{\mathbf{x}\} \cup \{\mathbf{x}^\mu\}$ 
9:   Update inhibitory weights:  $A_i \leftarrow A_i + \beta v_i(t_f)$ 
10:   $j \leftarrow j + 1$ 
11: end while
12: Return:  $\{\mathbf{x}\}$ 

```

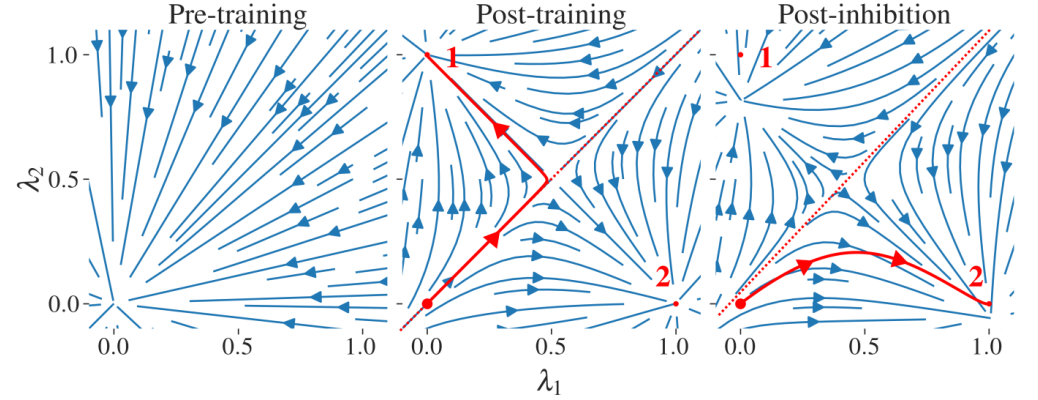


Fig 2. Stream plots of the CHN in a 2D subspace spanned by two pattern states. We examine a network with two stored patterns, labeled "1" and "2", and their stable states $\tilde{\mathbf{u}}^1$ and $\tilde{\mathbf{u}}^2$; these define a two-dimensional subspace parametrized by $\tilde{\mathbf{u}}(\lambda_1, \lambda_2) = \lambda_1 \tilde{\mathbf{u}}^1 + \lambda_2 \tilde{\mathbf{u}}^2$. At each point $\tilde{\mathbf{u}}(\lambda_1, \lambda_2)$ in this subspace, we compute the full N -dimensional time derivative via Eq. 4 and project it back onto the $\{\tilde{\mathbf{u}}^1, \tilde{\mathbf{u}}^2\}$ -plane to obtain the plotted flow. $\tilde{\mathbf{u}}^1$ and $\tilde{\mathbf{u}}^2$ have minimal correlation as they are generated from random binary patterns $\mathbf{x}^1$ and $\mathbf{x}^2 \in \{0, 1\}^N$. **(left)** Before training. **(middle)** After storing the two patterns using the GDA, each pattern state (1 and 2) becomes a stable attractor; the neutral state $\mathbf{u}_N = \mathbf{0}$ (large red dot) is on an unstable manifold. The red line corresponds to the trajectory of the network projected on the subspace. The trajectory follows the unstable manifold before arbitrarily falling into one of the two stored patterns. **(right)** Adaptation $\mathbf{A}$ is updated to apply enhanced inhibition to pattern 1 which shrinks its basin of attraction. This induce a shift in the position of the separatrix (red dotted line), favoring the flow to pattern 2. An exaggerated large inhibitory coefficient $\beta = 0.5$ is used to illustrate the modification of the vector field. Similar stream plots are obtained from networks storing patterns with various degrees of correlation using the GDA.

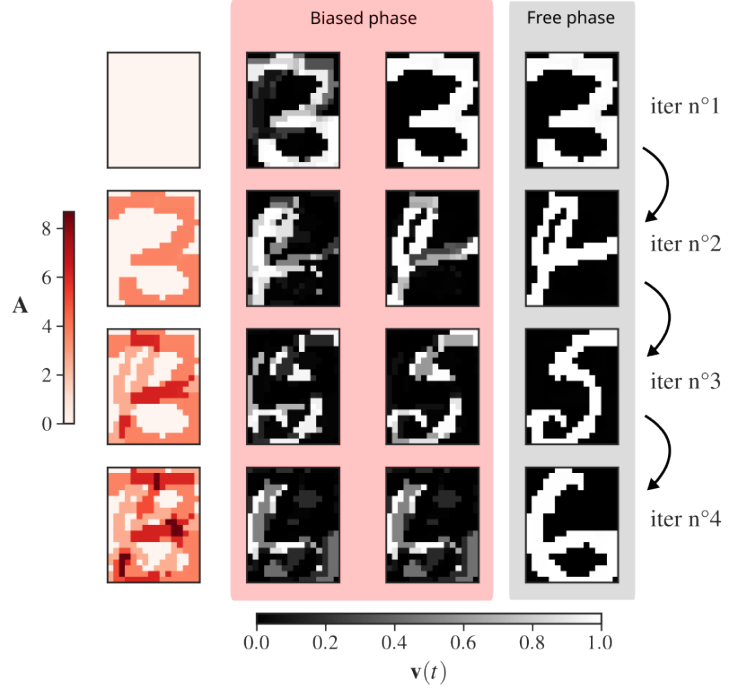


Fig 3. Recovery of three binary-patterns representations of handwritten digits from the MNIST dataset, “3”, “4”, “6”, “5”. The network is of size 20×16 as each neuron codes for a pixel. Left panels (in red): Evolution of adaptation of each neuron A_i . Right panels (pink and gray): snapshots of the evolution of the rates $v_i(t)$ for each unit of the network. Each row corresponds to the start of a new memory retrieval in the CHN. The free phase correspond to the removal of the inhibitory drive biasing the activity of each neurons as described in Alg. 3. The panels illustrate how the biased phase “orients” the evolution, and then the free phase provides the precise convergence to a stored pattern. An exaggerated large inhibitory coefficient $\beta = 2$ is used to illustrate the stable states deformation at the end of the biased phase, highlighting the need of the free phase.

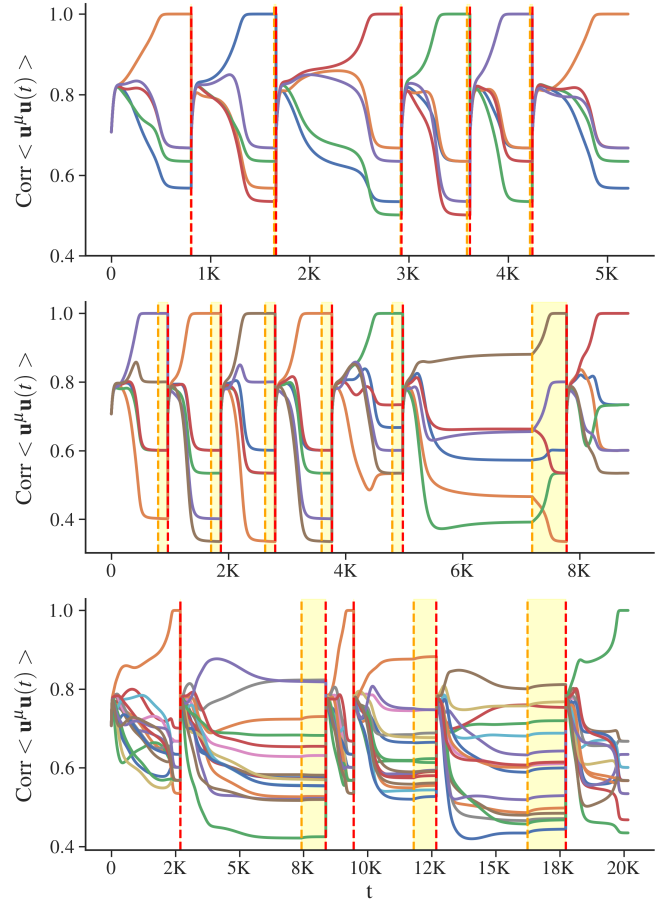


Fig 4. Visualization of AR (Alg. 3). The x-axis shows the number of time steps for simulations of the CHN, with all query iterations of AR concatenated. The y-axis shows the correlation between the state of the network and any stored state, with each color corresponding to a specific memory. Vertical red dashed lines separate successive memory retrieval sequences. At the beginning of a memory recall adaptation $\mathbf{A}$ is updated and the initial state is the neutral one. The yellow dashed lines indicate the end of the biased phase. Between yellow and red lines is the free phase, during which self-inhibition is removed, to converge precisely into a stored state. Considered patterns have minimal correlation as they are generated from random binary vectors. **(Top)** Recovery dynamics in a low-load network: 5 patterns stored in a network of 60 units. Each memory is visited once before all are successfully recovered. **(Middle)** Recovery dynamics in a critically loaded network: 5 patterns stored in a network of 30 units. Some memories are revisited multiple times before all are recovered. **(Bottom)** Recovery dynamics in a high-load network: 16 patterns stored in a network of 60 units. Recovery fails as spurious patterns are visited, reflected by low correlation values.

2 Results

In this section, we evaluate the ability of our algorithm to retrieve correlated patterns from a given network. We will consider the retrieval of pattern sets with various amounts of correlation. Each set is assigned a random binary parent pattern, where each bit is independently set to 0 or 1 with equal probability. Each pattern of a given set is generated by randomly choosing a fraction $(1 - \rho)$ of bits from the parent pattern and randomizing them. The randomization process sets each selected bit to 0 or 1 with equal probability, independently of its original value, as described in Alg. 5 (App.5.3).

Therefore, a higher ρ induces more correlation, while a lower ρ results in less correlation. Networks with various loads will be tested. All retrieval dynamics are considered without free phases as it has been observed that, for the retrieval of random binary patterns and the use of small inhibitory potentiation values β , the free phase does not significantly improve retrieval performance.

Fig. 5 illustrates, for various correlations, the loads for which AR allows systematic recovery of all stored patterns without external cues or memory lists. This property enables the continuous incorporation of new correlated memories, as described in Alg. 2. The retrieval improves with network size - larger networks can reliably retrieve more stored patterns without encountering false memories. We can observe the rather surprising feature that a moderate amount of correlation ($\rho \approx 0.5$) tends to improve pattern retrieval compared to highly correlated and minimally correlated pattern sets. In fact, this finding contrasts with traditional associative memory models, where correlation typically degrades performance [25].

Our interpretation is that an intermediate amount of correlation helps the system to be driven in the “good” direction in early stages of the evolution, i.e. while still rather close to the neutral state, and the energy landscape is rather featureless. Once driven to a point of the state space proximal to all the stored patterns, the system can finish the convergence. As illustrated in Appendix 5.5, Fig. 11, this push towards the good direction can be measured through the average correlation between the synaptic drive of each unit and the stored patterns.

As emerges from our simulations, to limit the appearance of false memories, the values of β must be relatively small compared to the target potential u_{target} . In our case, we found it convenient to keep β smaller than 0.1. By keeping the nudging of the convergence dynamic small, the emergence of false memories that might otherwise result from deformations in the energy landscape is reduced. Fig. 6 indicates the existence of a trade-off: Smaller β values require more iterations to retrieve all patterns but lower the chances of retrieving spurious patterns, while larger values accelerate pattern retrieval at the cost of increasing the probability of encountering a spurious state. Higher values of β than those considered in this study lead to catastrophic degradation of recovery dynamics for which no stored patterns are recovered. Lower values of β than presented here only lead to the need for more iterations to retrieve all patterns. For $\beta = 0$ the network converges toward one of the stored pattern states, and, so long we start near the same initial (neutral) state, it will always fall back into this attractor after each reset, we call this attractor the dominant attractor (or the dominant pattern) of the neutral state.

We shall now describe some qualitative information on the efficiency of our algorithm. As expected, the number of iterations required to successfully store patterns using Alg. 1 increases with the memory load (Fig. 7). Moreover, the higher the correlation between patterns, the higher the number of iterations needed for storage. Overall, our method requires significantly more iterations than previously documented for associative memory tasks in DHNs [7]. We can argue that this increase in computational cost may come from two factors. First, training a CHN through GDA (Alg. 1) inherently requires more iterations than training DHNs. Second, we observed that a smaller convergence

parameter ϵ in Alg. 1, while computationally more demanding, yields superior retrieval performance. We hypothesize that this tighter convergence criterion induces stronger competition between pattern attractors at the neutral state. This competition results in enhanced network responsiveness to subtle modifications in dynamics induced by adaptation during retrieval. These observations led to the adoption of a very small $\epsilon = 10^{-6}$, which demanded more iterations when performing the GDA.

Finally, Fig. 8 gives a visual summary of the continuous incorporation of new correlated memories allowed by our AR mechanism (Alg. 2). The whole learning scheme can be divided into two phases: an awake phase during which new patterns are collected, and a sleep phase during which previous patterns are retrieved using AR. The storage of patterns collected during both phases requires the use of the GDA. During this storage, the adaptation terms are turned off for the GDA to converge.

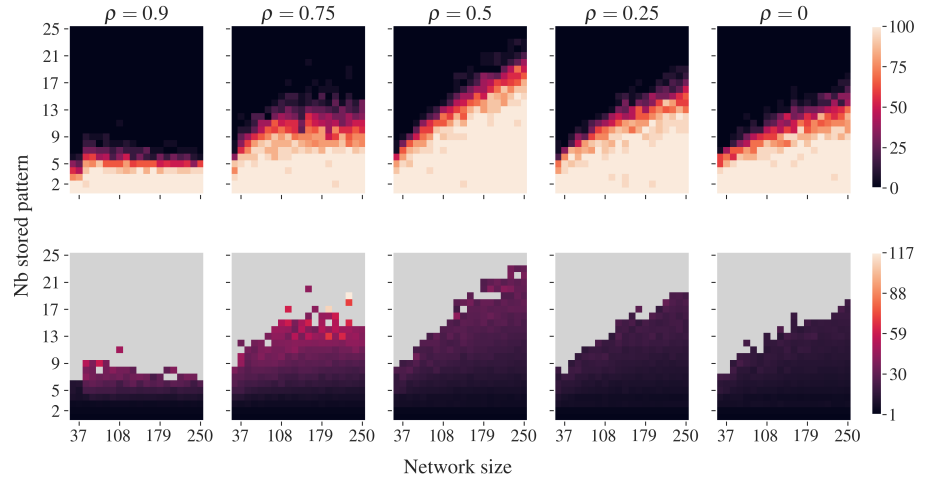


Fig 5. Recovery capacity of AR for various correlations, modulated by ρ . Data are averaged over 20 simulations with different pattern sets. Correlated patterns are generated using Alg. 5 (Appendix 5.3). For networks of various sizes and numbers of stored patterns, we employ Alg. 3 until all patterns are recovered or until a spurious state is found. **(Top row)** The percentage of ‘full retrieval,’ i.e., simulation runs where no spurious state occurs before recovering all stored patterns. **(Bottom row)** Number of iterations required to recover all patterns. Recovery is best for $\rho = 0.5$, which denotes equal number of correlated and uncorrelated bits. For small ρ values with few correlated bits, performance worsens. For highly correlated sets, only very low loads are recovered without false memories.

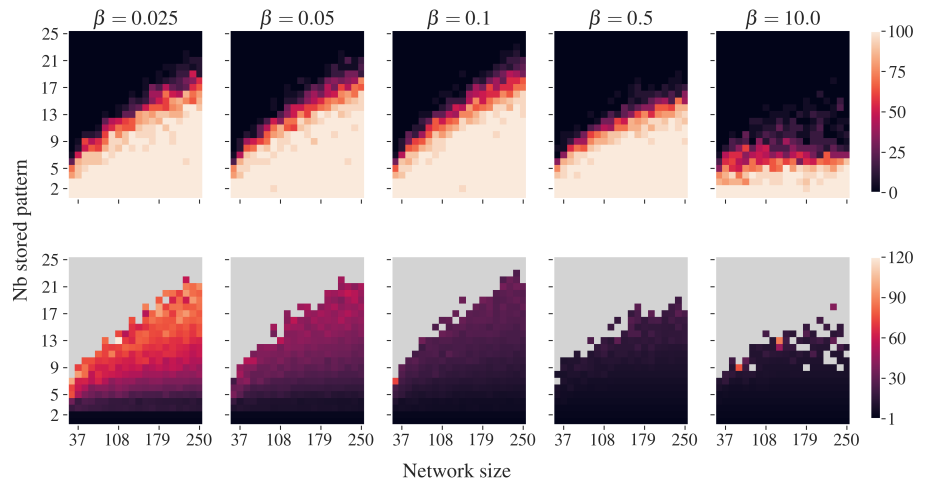


Fig 6. Similar to Fig. 5 but for various β values. Correlated patterns are generated with a ρ of 0.5. Lower β values require more iterations to recover the complete set of stored patterns. Higher β values diminish the number of iteration needed but increase the probability of finding spurious state when the load is important.

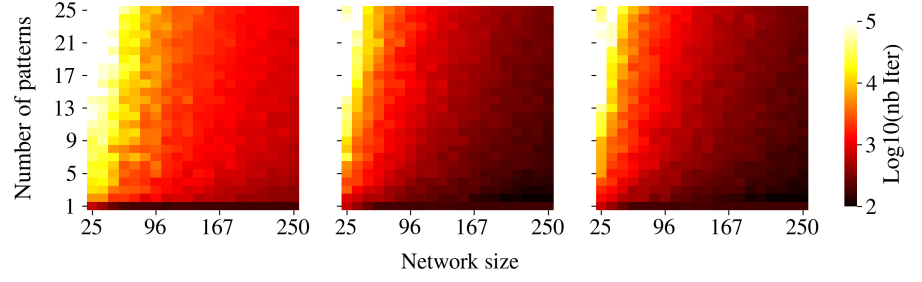


Fig 7. Convergence time of the GDA. Color intensity shows the number of iterations required to store a given number of patterns using GDA (Alg. 1). Data are averaged over 20 simulations for various pattern sets. ρ is equal to 0.5 for all simulations.

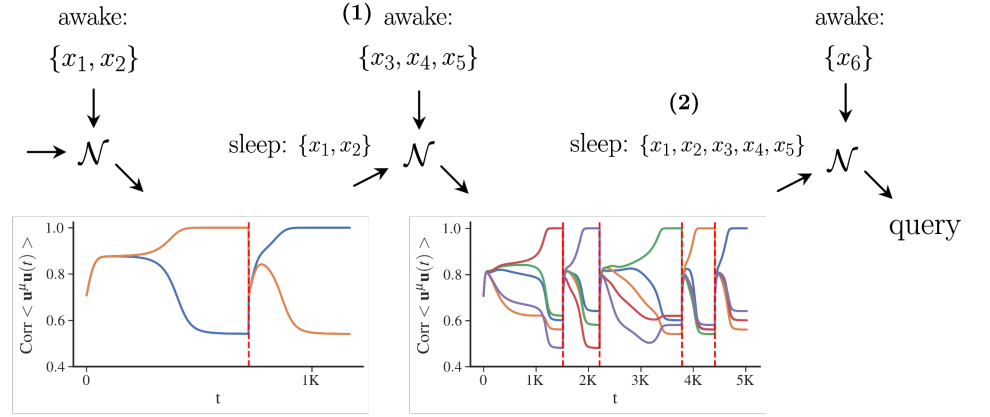


Fig 8. Continuous incorporation of correlated patterns. Visualization of Alg. 2. $\mathcal{N}$ denotes the network. The awake phase corresponds to the incorporation of new memories, while the sleep phase corresponds to AR. **(1)** Memories added during the second awake phase. **(2)** Memories recovered during the second sleep phase. Memories retrieved during sleep, along with newly encountered ones, are stored in the network using the GDA. At the end of the simulation, querying the network enables the retrieval of the six incrementally stored memories $\{x_1, \dots, x_6\}$.

3 Discussion

Our work introduces a biologically inspired mechanism for the continuous incorporation of correlated patterns in associative networks. CL is made possible through the autonomous recovery of all stored patterns during a retrieval phase. By systematically retrieving memories, the network can incorporate new patterns while mitigating the forgetting of the ones already stored. Autonomous retrieval is made possible by adaptation, avoiding the need to recall patterns from an external list. Our results indicate that larger networks experience fewer spurious state visits, suggesting improved reliability with scale. However, understanding how these dynamics extend to networks of biologically relevant sizes would require further investigation.

Traditional associative memory models typically suffer from decreased capacity when storing correlated patterns [16]. However, our findings revealed the rather unexpected feature that moderate correlation levels ($\rho = 0.5$) actually improve pattern retrieval in

the context of autonomous retrieval. We argue how this result may arise from the way that correlated structures influence the geometry of the attractor basins and the ensuing flow towards them. A more thorough theoretical analysis of this phenomenon could provide information on the factors that influence the robustness of recovery dynamics in both artificial and biological systems.

A key consideration is whether the success of the recovery dynamic is tightly dependent on the statistics of the stored patterns. The way patterns are generated using Alg. 5 results in half of the neurons being activated and half remaining inactive for each stored pattern on average. We can formalize it by defining ' a ' as the average number of activated units ($x_i = 1$), N as the network size, and $s = 1 - \frac{a}{N}$ as the sparsity, which yields $s = 0.5$ for the results presented above. To assess whether this specific sparsity is essential for the retrieval mechanism, we evaluated the recovery process using patterns with different sparsity levels. The results are presented in Appendix 5.6 and demonstrate that the retrieval mechanism remains robust to sparsity variations within the range $0.4 < s < 0.6$, but performance gradually degrades for values outside this range. Note that in this investigation, all patterns stored within each network share the same sparsity value. An interesting direction for future research would be to examine the network's ability to retrieve a set of patterns exhibiting varying degrees of sparsity.

presented here is robust to moderate amounts of noise, but it breaks down and induces spurious patterns when strong stochasticity is introduced. The impact of noise on the recovery dynamics has been tested by adding a stochastic term to the dynamics of each neuron:

$$c \frac{du_i}{dt} = \sum_j W_{ij} v_j - A_i v_i - \frac{u_i}{r} + D \eta_i(t), \quad (6)$$

where $\eta_i(t) \sim \mathcal{U}(-1, 1)$ and $D < 1$ is the noise scaling factor. This added stochasticity does not alter the recovery dynamics for $D < 0.1$ but induces spurious patterns for higher values. Importantly, this stochasticity does not by itself induce the exploration of various memory states when $\beta = 0$. We have also tested slightly randomizing the neutral state, with $u_i(0) = D \eta_i$. This approach yields results identical to adding stochasticity to neural dynamics: convergence remains unchanged for $D < 0.1$, while spurious patterns emerge at higher values. This suggests that, even though the neutral state lies near the separatrix, in high dimensions, the dominance of an attractor cannot be easily disrupted through stochasticity alone, at least for the continuous model and learning scheme considered here. While this statement may depend on the specific values of N and D , a quantitative exploration of this feature is beyond the scope of the present work.

Previous work in computational neuroscience indicates that inhibitory circuits may play a critical role in the regulation of neural activity and plasticity [14, 26]. Here we demonstrate that inhibitory plasticity or SFA could be one of the key mechanisms that allows the sequential reactivation of memories observed during sleep and resting states [4, 27]. A property of associative networks highlighted by our approach is that subtle changes in self-inhibition can drive substantial shifts in network dynamics without disrupting the fundamental structure of stored attractors. Inhibition, therefore, allows for context-dependent activity of the network as observed in experimental settings [28]. Biological neural circuits might, therefore, employ similar mechanisms to navigate complex, correlated memory spaces. In our model, adaptation, or self-inhibition, is only potentiated at the end of each memory replay. This behavior contrasts with the plasticity dynamics in the brain, which are considered to occur continuously. This schematic implementation is intended to demonstrate the functional role of inhibition in guiding memory exploration, rather than to reproduce the precise temporal dynamics of neuronal plasticity. Another discontinuity in the plasticity dynamics comes from the fact that, during the training of the network on a new memory corpus, self-inhibition has to be removed to avoid disturbance of the storage

process. Implementing a model that more closely matches the continuous dynamics
observed in the brain would therefore represent a natural extension of this work.

The ability of our network to transition from one memory state to another critically
depends on the reset of the dynamics to the neutral state. In the brain, this reset
mechanism may likely be induced by observed physiological processes. During sleep, for
instance, cortical neurons exhibit bistable dynamics, alternating between depolarized UP
states characterized by sustained firing and hyperpolarized DOWN states with neuronal
silence [29–31]. These UP states, lasting hundreds of milliseconds to seconds, provide
windows for memory replay, while DOWN states could naturally separate distinct
memory activations. Another natural extension of our work would be to incorporate
specific neural mechanisms responsible for such network resets, such as post-inhibitory
rebound, to implement a more biologically plausible version of our model.

4 Conclusion

In this work, we demonstrate how adaptation can enable autonomous exploration of attractor landscapes in continuous Hopfield networks. Our key finding reveals that, under a critical load, inhibitory plasticity allows networks to systematically retrieve the entire set of stored memories, even for highly correlated sets. This property allowed us to propose and test an algorithmic scheme for continuous learning leveraging the ability of gradient descent to store correlated patterns. The capacity for self-directed pattern exploration, emerging from inhibitory modulation, offers insights for both biological memory consolidation and neuromorphic computing.

5 Appendix

5.1 Derivation of the gradient descent learning rule

At steady state, when $\frac{du_i}{dt} = 0$, from Eq. 1 we have:

$$u_i = r \sum_j W_{ij} v_j$$

where $v_j = \sigma(u_j)$. We can then define

$$\hat{u}_i = r \sum_j W_{ij} \sigma(\tilde{u}_j)$$

the expected potential of the unit i when the rest of the units are at their target potential $\tilde{u}_j$.

We can then define the error function E as the sum of squared differences between the expected potentials $\hat{u}_i$ and target potentials $\tilde{u}_i$:

$$E = \frac{1}{2} \sum_i (\tilde{u}_i - \hat{u}_i)^2$$

Taking the derivative with respect to the expected potential:

$$\frac{\partial E}{\partial \hat{u}_i} = (\tilde{u}_i - \hat{u}_i)$$

To compute how the error changes with respect to the weights W_{ij} , we use the chain rule:

$$\frac{\partial E}{\partial W_{ij}} = \frac{\partial E}{\partial \hat{u}_i} \frac{\partial \hat{u}_i}{\partial W_{ij}}$$

The partial derivative with respect to W_{ij} is:

$$\frac{\partial \hat{u}_i}{\partial W_{ij}} = r v_j$$

This is an approximation that captures the immediate direct effect of W_{ij} on $\hat{u}_i$, assuming that v_j remains constant and ignoring feedback effects in the recurrent network. This approximation is valid when the optimization steps are small relative to the nonlinearities in the sigmoid function.

Therefore, the error gradient with respect to W_{ij} becomes:

$$\frac{\partial E}{\partial W_{ij}} = (\tilde{u}_i - \hat{u}_i) r v_j$$

Finally, we update the weights using gradient descent:

$$\Delta W_{ij} = \alpha \frac{\partial E}{\partial W_{ij}} = \alpha r(\tilde{u}_i - \hat{u}_i)v_j$$

where α is the learning rate, typically set to 0.001 to ensure convergence.

5.2 Querying procedure

The network's ability to retrieve stored patterns can be tested through partial cues. Given a subset of informed units from a pattern μ , we initialize their potentials according to the partial pattern while leaving other units at rest :

$$u_i^\mu(t=0) = \begin{cases} +6 & \text{if unit } i \text{ is informed and } x_i^\mu = 1 \\ -6 & \text{if unit } i \text{ is informed and } x_i^\mu = 0 \\ 0 & \text{otherwise} \end{cases}$$

The network then evolves according to Eq. 1 until it reaches a stable state ($\|\frac{d\mathbf{u}}{dt}\|_\infty < \epsilon$). The thresholding procedure then allows for the reading of a stored binary pattern $\mathbf{x}^\mu$:

$$x_i^\mu = \begin{cases} 1 & \text{if } v_i(t_f) > 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

with $v_i(t_f)$ the rate of neuron i after convergence of the dynamic. Alg. 4 summarize the procedure.

When operating below capacity, the dynamics typically converges to the stored pattern most similar to the initial cue. The final state is interpreted as a binary pattern using the thresholding procedure (Eq.3). As with traditional DHNs, retrieval success depends both on the network load and on the number of units informed in the initial cue. Although a theoretical analysis of the storage capacity under Alg. 1 would be of interest, our focus here is on the autonomous retrieval mechanism, which operates in a regime well below this limit for which pattern retrieval is highly reliable.

Algorithm 4 Querying with convergence of the network using Euler method

- 1: Given a pattern μ and a subset of informed units
 - 2: **set** $u_i^\mu(t=0) = \begin{cases} +u_{\text{target}} & \text{if } x_i^\mu = 1 \\ -u_{\text{target}} & \text{if } x_i^\mu = 0 \\ 0 & \text{if } x_i^\mu \text{ not informed} \end{cases}$
 - 3: **repeat**
 - 4: **for** each unit i **do**
 - 5: $u_i(t+1) = u_i(t) + \delta(\sum_j W_{ij}v_j - \frac{u_i(t)}{\tau})$
 - 6: **end for**
 - 7: **until** $\|\dot{\mathbf{u}}\|_\infty < \epsilon$
 - 8: **define** $t_f = t$ ▷ Final time at convergence
 - 9: **read** : $x_i^\mu = \begin{cases} 1 & \text{if } v_i(t_f) > 0.5 \\ 0 & \text{otherwise.} \end{cases}$
-

5.3 Construction of correlated patterns

425

Algorithm 5 Generation of p correlated random patterns

```

1: Generate a random binary pattern  $\mathbf{x}^{parent} \in \{0, 1\}^N$ 
2: Set  $k = \lfloor (1 - \rho)N \rfloor$ , where  $\rho$  is the ratio of bits not randomized
3: for  $i = 1$  to  $p$  do
4:    $\mathbf{x}^i \leftarrow \mathbf{x}^{parent}$ 
5:   Randomly select  $k$  distinct indices  $\{j_1, \dots, j_k\}$  from  $\{1, \dots, N\}$ 
6:   for  $m = 1$  to  $k$  do
7:      $x_{j_m}^i \leftarrow$  random choice from  $\{0, 1\}$  with equal probability
8:   end for
9: end for

```

5.4 Model with inhibitory matrix

426

To allow the sequential exploration of stored patterns a second approach have been tested. A set of plastic inhibitory synapses W'_{ij} has been added to the network dynamics :

427

428

429

$$c \frac{du_i}{dt} = \sum_j W_{ij} v_j - \sum_j W'_{ij} v_j - \frac{u_i}{r} \quad (7)$$

$$W'_{ij} \leftarrow W'_{ij} + \beta v_i(t_f) v_j(t_f) \quad (8)$$

430

with $v_i(t_f)$ the rate of neuron i after convergence of the dynamics. Self-inhibition $W'_{i,i}$ is set to 0 and undergoes no plasticity. A model combining self-inhibition and plastic inhibitory synapses has been tested and yields the same qualitative results. β represents the potentiation strength of the inhibitory synapses and is selected as a small value, typically below 0.05. The plasticity rule in Eq.8 is Hebbian.

431

432

433

434

435

We can observe that, similarly to adaptation, plastic synapses allow the retrieval of correlated memories Fig. 9 and 10. The potentiation strength β has to be weaker than that for adaptation, as a neuron is contacted by many inhibitory synapses. The retrieval dynamics are similar to the ones observed with adaptation in every aspect.

436

437

438

439

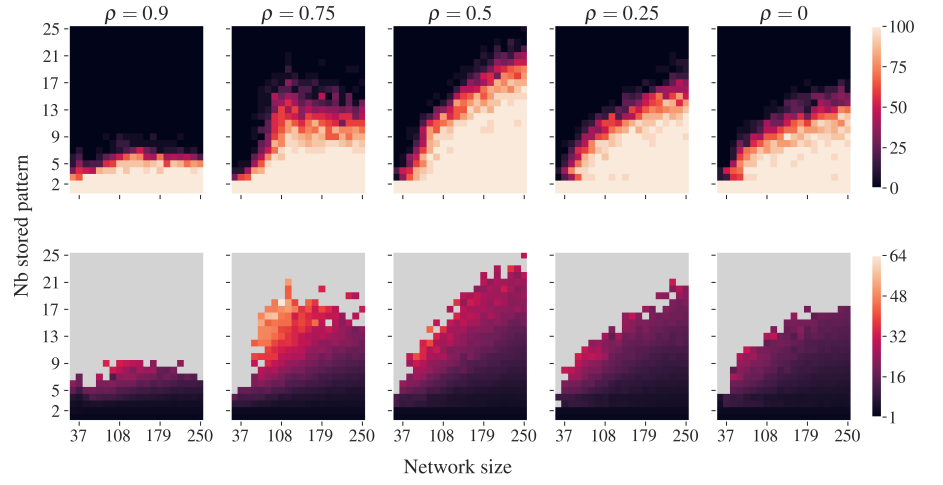


Fig 9. Similar to Fig. 5, recovery capacity of AR for various correlations but for a model using an inhibitory matrix $\mathbf{W}'$.

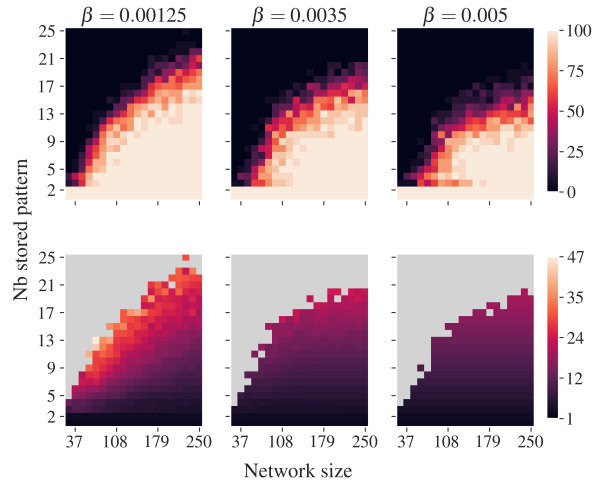


Fig 10. Similar to Fig. 6, recovery capacity of AR for various values of β but for a model using an inhibitory matrix $\mathbf{W}'$.

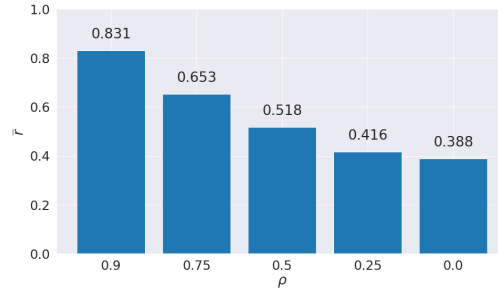


Fig 11. For networks trained with sets of varying correlation ρ , we compute the synaptic drive of each unit $I_j^{syn} = \sum_i W_{ij}$. This synaptic drive is the main component influencing the dynamic of the network at the neutral state, when the leak is null. Then, we compute the average correlation between each stored pattern and the synaptic drive: $\bar{r} = \frac{1}{p} \sum_{\mu=1}^p r^\mu$ with $r^\mu = \text{Corr}(\mathbf{x}^\mu, \mathbf{I}^{syn})$. We can see that, the higher the ρ , the stronger the average correlation between the drive $\mathbf{I}^{syn}$ and stored patterns. At the neutral state, from which the network is initialized, stronger correlations induce a stronger push of the network state toward stored patterns. This could explain why our algorithm performs better with moderately correlated patterns.

Algorithm 6 Generation of p correlated sparse binary patterns

```

1: Input: sparsity  $s$ , correlation parameter  $\rho$ 
2: Generate a binary parent pattern  $\mathbf{x}^{\text{parent}} \in \{0, 1\}^N$  with  $p(x_i = 0) = s$ 
3: Set  $k = \lfloor (1 - \rho)N \rfloor$ 
4: for  $i = 1$  to  $p$  do
5:    $\mathbf{x}^i \leftarrow \mathbf{x}^{\text{parent}}$ 
6:   Randomly select  $k$  distinct indices  $\{j_1, \dots, j_k\}$  from  $\{1, \dots, N\}$ 
7:   for  $m = 1$  to  $k$  do
8:      $x_{j_m}^i \leftarrow 0$  with probability  $s$ , otherwise 1
9:   end for
10: end for

```

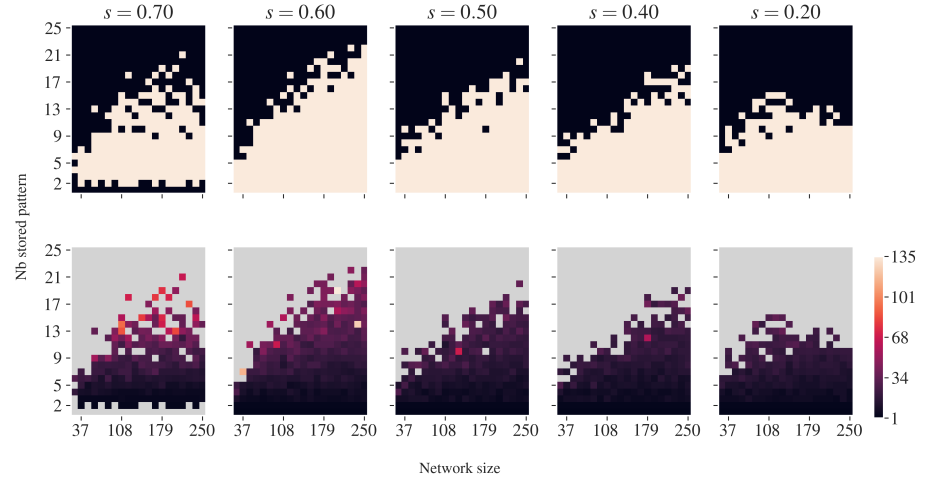


Fig 12. Similar to Fig. 5, this figure shows the recovery capacity of AR for various sparsity levels of the stored patterns. The data presented correspond to the results of AR for one simulation for each network size/number of stored patterns combination. **(Top row)** White pixels correspond to successful recovery of the full pattern set, while black pixels indicate the appearance of spurious patterns before full recovery is achieved. The appearance of spurious patterns for $s < 0.7$ with only one stored pattern is unexpected and necessitates further investigation.

5.7 Parameters for simulations

442

Table 1. CHN parameters

Name	Value	Description
ϵ_{sim}	10^{-6}	Convergence constant
r	1	Leak time constant
c	1	Integration time constant
δ	0.001	Euler step size

Table 2. Gradient descent

Name	Value	Description
α	0.0001	Learning rate
ϵ_{learn}	10^{-6}	Convergence constant
u^{target}	6	Target potential winning units

6 Acknowledgments

443

This project has received financial support from the French DIM AI4IDF and the ANR “MemAI” project ANR-23- CE30-0040. This work was supported by Paris Region.

444

445

References

1. Robins A. Catastrophic Forgetting, Rehearsal and Pseudorehearsal. *Connection Science*. 1995;7(2):123–146. doi:10.1080/09540099550039318.
2. McCALLUM S. Catastrophic Forgetting and the Pseudorehearsal Solution in Hopfield-type Networks. *Connection Science*. 1998;10(2):121–135. doi:10.1080/095400998116530.
3. Louie K, Wilson MA. Temporally Structured Replay of Awake Hippocampal Ensemble Activity during Rapid Eye Movement Sleep. *Neuron*. 2001;29(1):145–156. doi:10.1016/S0896-6273(01)00186-6.
4. Peyrache A, Khamassi M, Benchenane K, Wiener SI, Battaglia FP. Replay of rule-learning related neural patterns in the prefrontal cortex during sleep. *Nature Neuroscience*. 2009;12(7):919–926. doi:10.1038/nn.2337.
5. Fauth MJ, Van Rossum MC. Self-organized reactivation maintains and reinforces memories despite synaptic turnover. *eLife*. 2019;8:e43717. doi:10.7554/eLife.43717.
6. Tononi G, Cirelli C. Sleep and the Price of Plasticity: From Synaptic and Cellular Homeostasis to Memory Consolidation and Integration. *Neuron*. 2014;81(1):12–34. doi:10.1016/j.neuron.2013.12.025.
7. Diederich S, Oppen M. Learning of correlated patterns in spin-glass networks by local learning rules. *Physical Review Letters*. 1987;58(9):949–952. doi:10.1103/PhysRevLett.58.949.
8. Haxby JV, Gobbini MI, Furey ML, Ishai A, Schouten JL, Pietrini P. Distributed and Overlapping Representations of Faces and Objects in Ventral Temporal Cortex. *Science*. 2001;293(5539):2425–2430. doi:10.1126/science.1063736.
9. Kriegeskorte N. Representational similarity analysis – connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*. 2008;doi:10.3389/neuro.06.004.2008.
10. Quiroga RQ, Reddy L, Kreiman G, Koch C, Fried I. Invariant visual representation by single neurons in the human brain. *Nature*. 2005;435(7045):1102–1107. doi:10.1038/nature03687.
11. Kothapalli V. Neural Collapse: A Review on Modelling Principles and Generalization; 2023. Available from: <http://arxiv.org/abs/2206.04041>.
12. Hopfield JJ. Neurons with graded response have collective computational properties like those of two-state neurons. *Proceedings of the National Academy of Sciences*. 1984;81(10):3088–3092. doi:10.1073/pnas.81.10.3088.
13. Amit DJ, Gutfreund H, Sompolinsky H. Storing Infinite Numbers of Patterns in a Spin-Glass Model of Neural Networks. *Physical Review Letters*. 1985;55(14):1530–1533. doi:10.1103/PhysRevLett.55.1530.
14. Vogels TP, Sprekeler H, Zenke F, Clopath C, Gerstner W. Inhibitory Plasticity Balances Excitation and Inhibition in Sensory Pathways and Memory Networks. *Science*. 2011;334(6062):1569–1573. doi:10.1126/science.1211095.

15. Hopfield JJ. Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*. 1982;79(8):2554–2558. doi:10.1073/pnas.79.8.2554.
16. Amit DJ, Gutfreund H, Sompolinsky H. Spin-glass models of neural networks. *Physical Review A*. 1985;32(2):1007–1018. doi:10.1103/PhysRevA.32.1007.
17. McCloskey M, Cohen NJ. Catastrophic Interference in Connectionist Networks: The Sequential Learning Problem. In: *Psychology of Learning and Motivation*. vol. 24. Elsevier; 1989. p. 109–165. Available from: <https://linkinghub.elsevier.com/retrieve/pii/S0079742108605368>.
18. Kirkpatrick J, Pascanu R, Rabinowitz N, Veness J, Desjardins G, Rusu AA, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*. 2017;114(13):3521–3526. doi:10.1073/pnas.1611835114.
19. Shen Y, Dasgupta S, Navlakha S. Reducing Catastrophic Forgetting With Associative Learning: A Lesson From Fruit Flies. *Neural Computation*. 2023;35(11):1797–1819. doi:10.1162/neco_a01615.
20. Ha GE, Cheong E. Spike Frequency Adaptation in Neurons of the Central Nervous System. *Experimental Neurobiology*. 2017;26(4):179–185. doi:10.5607/en.2017.26.4.179.
21. Peron S, Gabbiani F. Spike frequency adaptation mediates looming stimulus selectivity in a collision-detecting neuron. *Nature Neuroscience*. 2009;12(3):318–326. doi:10.1038/nn.2259.
22. Kodangattil JN, Dacher M, Authement ME, Nugent FS. Spike timing-dependent plasticity at GABAergic synapses in the ventral tegmental area. *The Journal of Physiology*. 2013;591(19):4699–4710. doi:10.1113/jphysiol.2013.257873.
23. D’amour J, Froemke R. Inhibitory and Excitatory Spike-Timing-Dependent Plasticity in the Auditory Cortex. *Neuron*. 2015;86(2):514–528. doi:10.1016/j.neuron.2015.03.014.
24. LeCun Y, Cortes C, J C Burges C. The MNIST database of handwritten digits; 1998. Available from: <http://yann.lecun.com/exdb/mnist/>.
25. Fontanari JF, Theumann WK. On the storage of correlated patterns in Hopfield’s model. *Journal de Physique*. 1990;51(5):375–386. doi:10.1051/jphys:01990005105037500.
26. Barron HC, Vogels TP, Emir UE, Makin TR, O’Shea J, Clare S, et al. Unmasking Latent Inhibitory Connections in Human Cortex to Reveal Dormant Cortical Memories. *Neuron*. 2016;90(1):191–203. doi:10.1016/j.neuron.2016.02.031.
27. Wilson MA, McNaughton BL. Reactivation of Hippocampal Ensemble Memories During Sleep. *Science*. 1994;265(5172):676–679. doi:10.1126/science.8036517.
28. Kuchibhotla KV, Gill JV, Lindsay GW, Papadoyannis ES, Field RE, Sten TAH, et al. Parallel processing by cortical inhibition enables context-dependent behavior. *Nature Neuroscience*. 2017;20(1):62–71. doi:10.1038/nn.4436.
29. Jercog D, Roxin A, Barthó P, Luczak A, Compte A, De La Rocha J. UP-DOWN cortical dynamics reflect state transitions in a bistable network. *eLife*. 2017;6. doi:10.7554/elife.22425.

30. Luczak A, Barthó P, Marguet SL, Buzsáki G, Harris KD. Sequential structure of neocortical spontaneous activity *in vivo*. Proceedings of the National Academy of Sciences. 2007;104(1):347–352. doi:10.1073/pnas.0605643104.
31. Steriade M, Nuiiez A, Amzica F. A Novel Slow (<1 Hz) Oscillation of Neocortical Neurons in viva: Depolarizing and Hyperpolarizing Components;.